SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	N.kalyan Reddy	**Reg. No.:**	192424232
Date:	13-08-2026	**Duration:**	60 minutes

Code of Conduct

I N.KALYAN REDDY (192424232) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
2. **Project Structure Design**
   - Design and explain the folder structure of your project repository.
   - Justify the purpose of each major folder and file included in the repository.
3. **Git Repository Initialization**

    - Create a Git repository for the project.
    - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy**
    - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
    - Justify why the selected branching model is appropriate for the assigned project.
5. **Version Control Workflow**
    - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
    - Explain the purpose of each Git operation performed.
6. **Commit History Analysis**
    - Present the commit history of the project.
    - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
    - Explain why Docker is suitable for the assigned project.
    - Identify the application components that require containerization.
8. **Dockerfile Development**
    - Design and explain the Dockerfile required to containerize the application.
    - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
    - Develop a Docker Compose configuration for the project.
    - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
    - Demonstrate the execution of the application using Docker containers.
    - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
    - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
    - Identify the challenges encountered during implementation.
    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution

- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

%)	improvements.			

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

1. Problem Analysis

Problem Statement

The Intelligent AI-Based Smart Parking and Vehicle Guidance System is developed to solve parking shortages and traffic congestion in metropolitan areas. The system uses AI, IoT sensors, cameras, GPS, and cloud computing to provide real-time parking information and improve urban mobility.

Drivers often spend considerable time searching for vacant parking spaces. This increases traffic congestion, fuel consumption, and carbon emissions. The proposed system detects vacant parking spaces, guides drivers to available spaces,

predicts parking demand, supports parking reservations, provides digital payment facilities, and generates parking occupancy analytics.

Project Objectives

1. Detect parking availability automatically.
2. Guide drivers to vacant spaces.
3. Predict parking demand using AI.
4. Automate parking reservations.
5. Support digital payment integration.
6. Monitor parking occupancy.
7. Generate parking utilization reports.
8. Reduce urban traffic congestion.

Functional Requirements

1. The system shall detect vacant and occupied parking spaces.
2. The system shall provide real-time parking availability.
3. The system shall guide drivers to vacant parking spaces.
4. The system shall predict parking demand using AI.
5. The system shall allow users to reserve parking spaces.
6. The system shall support digital payments.
7. The system shall monitor parking occupancy.
8. The system shall generate parking utilization reports.

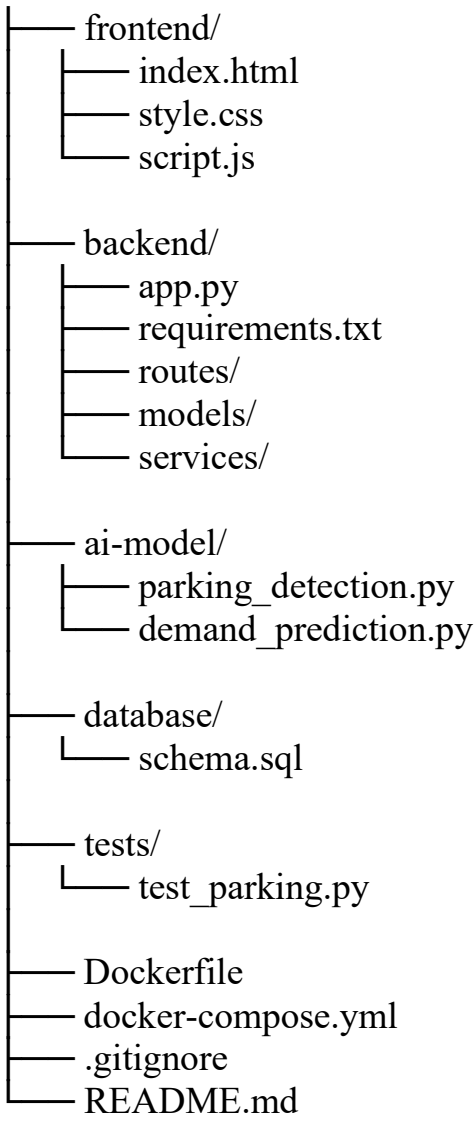
Expected Outcomes

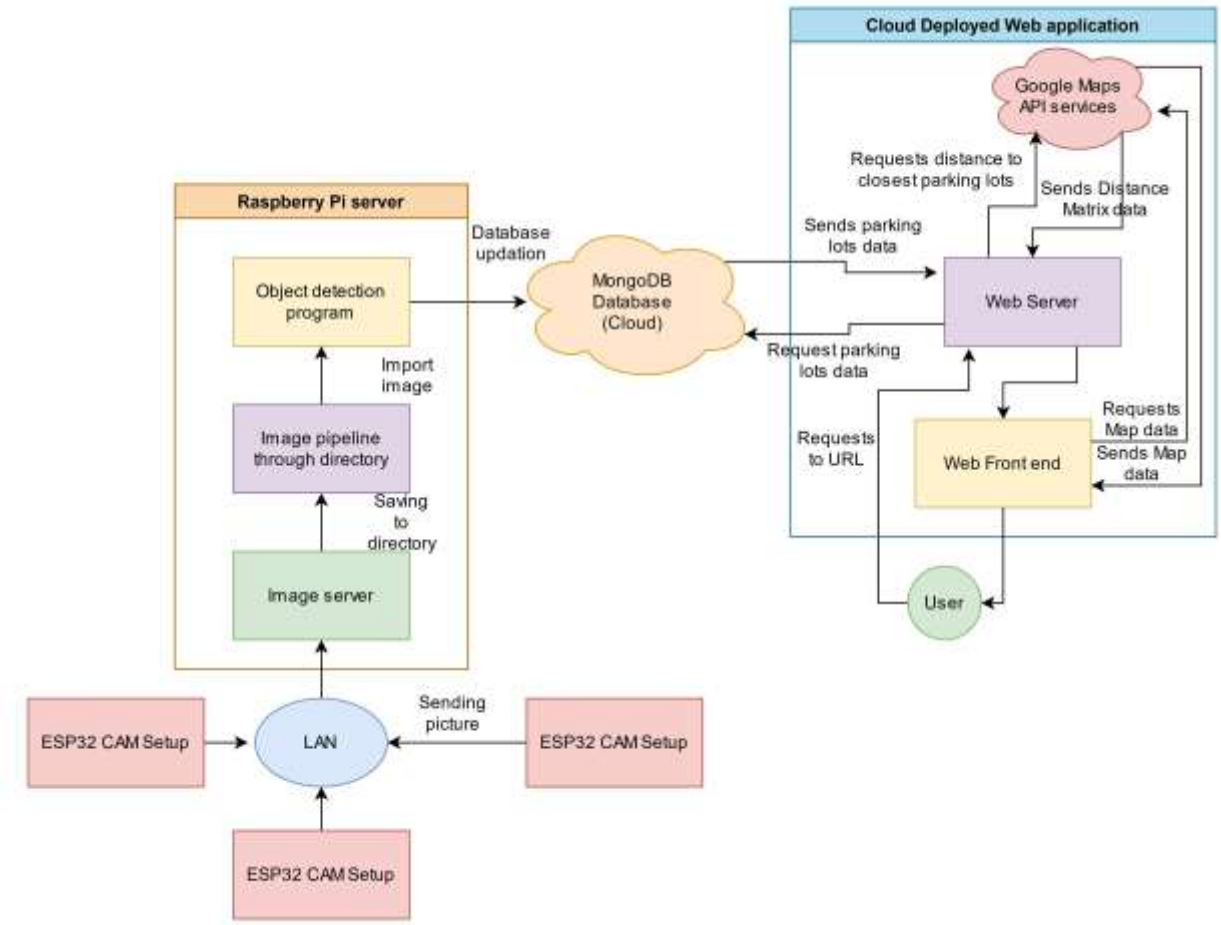
1. Reduced parking search time.
2. Lower traffic congestion.
3. Reduced fuel consumption.
4. Improved parking utilization.
5. Enhanced user convenience.
6. Increased parking revenue.
7. Better urban mobility.
8. Reduced carbon emissions.

2. Project Structure Design

Folder Structure

smart-parking-system/


frontend/
index.html
style.css
script.js
backend/
app.py
requirements.txt
routes/
models/
services/
ai-model/
parking_detection.py
demand_prediction.py
database/
schema.sql
tests/
test_parking.py
Dockerfile
docker-compose.yml
.gitignore
README.md



Purpose of Major Folders and Files

- frontend/ – Contains the user interface of the smart parking system.
- backend/ – Contains the server-side application and APIs.
- ai-model/ – Contains AI-based parking detection and demand prediction programs.
- database/ – Contains database-related files.
- tests/ – Contains testing programs.
- Dockerfile – Defines the instructions for creating the application container.
- docker-compose.yml – Defines the services required to run the application.
- .gitignore – Specifies files that should not be tracked by Git.
- README.md – Contains project information and instructions.

3. Git Repository Initialization

Git Repository Creation

The Git repository is created using the following commands:

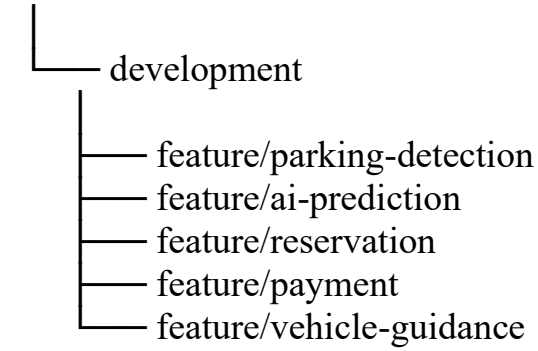
```
mkdir smart-parking-system
cd smart-parking-system
git init
git add .
git commit -m "Initial project setup"
git branch -M main
git remote add origin <repository-url>
git push -u origin main
```

Purpose of Essential Git Files

- .git/ – Stores Git repository information, branches, and commit history.
- .gitignore – Prevents unnecessary files from being tracked.
- README.md – Provides information and instructions about the project.

4. Git Branching Strategy

The project uses the following branching strategy:



Branch Description

- Main branch – Contains the stable version of the project.
- Development branch – Used for integrating ongoing development work.
- Feature branches – Used to develop individual project features.
- Release branch – Used for preparing a stable release.
- Hotfix branch – Used to fix critical problems in the application.

Justification

This branching strategy is appropriate because the Smart Parking System contains different features such as parking detection, AI prediction, reservation, payment, and vehicle guidance. Separate feature branches allow each feature to be developed and tested independently before being merged into the development and main branches.

5. Version Control Workflow

Branch Creation

```
git checkout development
git checkout -b feature/parking-detection
```

Add and Commit Changes

```
git add .
git commit -m "Add parking detection feature"
```

Push the Branch

```
git push origin feature/parking-detection
```

Merge the Branch

```
git checkout development
git merge feature/parking-detection
```

Conflict Resolution

If a merge conflict occurs:

```
git status
```

The conflicting file is edited to resolve the conflict. After resolving it:

```
git add .
git commit -m "Resolve merge conflict"
```

Repository Synchronization

```
git pull origin development
git push origin development
```

Purpose of Git Operations

- git init – Initializes the Git repository.
- git add – Stages project changes.

- git commit – Saves changes in Git history.
- git checkout – Switches between branches.
- git merge – Combines changes from different branches.
- git pull – Downloads changes from the remote repository.
- git push – Uploads local changes to the remote repository.

6. Commit History Analysis

Sample Commit History

Initial project setup
Add frontend parking interface
Add parking detection module
Add AI demand prediction
Add parking reservation feature
Add digital payment module
Add vehicle guidance feature
Add Dockerfile
Add Docker Compose configuration
Fix parking availability calculation

Commit Message Practices

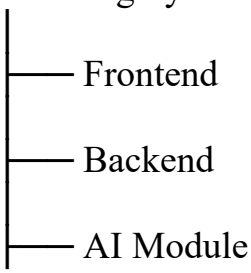
Commit messages are written clearly and describe the specific changes made to the project. Meaningful commit messages make the project history easier to understand and help in project maintenance and troubleshooting.

7. Docker Environment Design

Docker is suitable for the Smart Parking System because the project contains multiple application components that need a consistent environment for development, testing, and deployment.

Components Requiring Containerization

Smart Parking System



Docker Components

- Frontend container – Runs the user interface.
- Backend container – Runs the application server and APIs.
- AI container – Runs parking detection and demand prediction.
- Database container – Stores parking, user, reservation, and occupancy data.

8. Dockerfile Development

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY backend/requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY backend/ .

EXPOSE 5000

CMD ["python", "app.py"]
```

Purpose of Dockerfile Instructions

- FROM – Selects the Python base image.
- WORKDIR – Sets the working directory inside the container.
- COPY – Copies required project files into the container.
- RUN – Installs the required Python packages.
- EXPOSE – Specifies the port used by the application.
- CMD – Starts the application.

9. Docker Compose Design

docker-compose.yml

```
services:
```

```
  backend:
    build: .
    ports:
      - "5000:5000"
    environment:
      - DATABASE_URL=postgresql://parking:password@database:5432/parkingd
b
    depends_on:
      - database

  frontend:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:
      - ./frontend:/usr/share/nginx/html

  database:
    image: postgres:15
    environment:
      POSTGRES_USER: parking
      POSTGRES_PASSWORD: password
      POSTGRES_DB: parkingdb
    volumes:
      - parking_data:/var/lib/postgresql/data

volumes:
  parking_data:
```

Services

- Backend – Provides application logic and APIs.
- Frontend – Provides the user interface.
- Database – Stores system data.

Networking

Docker Compose allows the services to communicate with each other through the Docker network.

Volumes

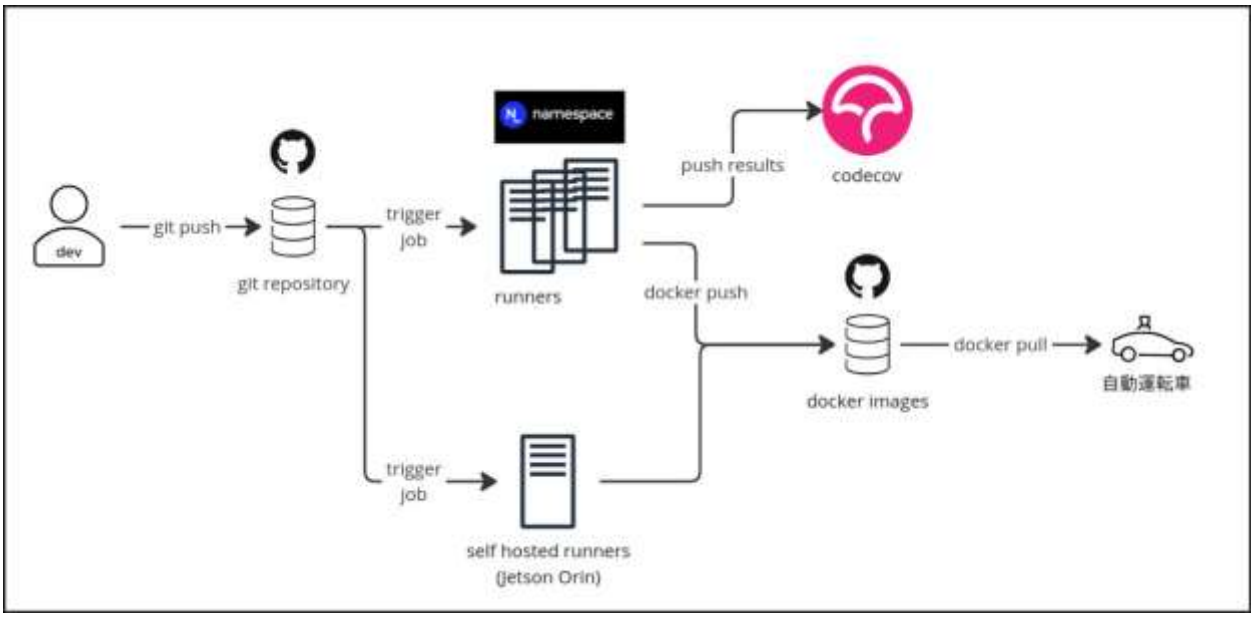
The parking_data volume stores database information and maintains the data when the database container is restarted.

Environment Variables

Environment variables provide database configuration information such as the database username, password, and database name.

Dependencies

The backend depends on the database service for storing and retrieving application data.



10. Container Deployment and Testing

Build the Application

docker compose build

Start the Containers

docker compose up -d

Check Running Containers

docker compose ps

View Container Logs

docker compose logs

Test the Application

The frontend can be accessed through:

http://localhost:8080

The backend can be accessed through:

http://localhost:5000

Testing Procedure

1. Build the Docker images.
2. Start the containers.
3. Check whether all containers are running.
4. Open the Smart Parking application.
5. Check parking availability.
6. Test parking reservation.
7. Verify backend connectivity.
8. Verify database connectivity.
9. Confirm that the application runs successfully.

Stop the Containers

docker compose down

11. Benefits of Git and Docker

Git Benefits

- Improves collaboration between developers.
- Maintains project version history.
- Supports branching and merging.
- Makes changes easy to track.
- Helps maintain the project.

Docker Benefits

- Provides application portability.

- Simplifies deployment.
- Provides a consistent environment.
- Supports scalability.
- Simplifies application maintenance.
- Reduces dependency-related problems.

Git and Docker together improve collaboration, version management, portability, deployment, scalability, and maintainability of the Smart Parking System.

12. Challenges and Improvements

Challenges

1. Accurate detection of vacant parking spaces.
2. Handling real-time parking occupancy data.
3. Integrating camera and IoT sensor data.
4. Maintaining synchronization between parking availability and reservations.
5. Managing communication between Docker containers.
6. Resolving Git merge conflicts.
7. Managing Docker dependencies and configurations.

Improvements and Future Enhancements

1. Improve AI-based parking demand prediction.
2. Improve camera-based parking detection accuracy.
3. Integrate real-time GPS navigation.
4. Add advanced digital payment facilities.
5. Implement automatic vehicle identification.
6. Improve cloud-based deployment.
7. Add real-time parking notifications.
8. Improve scalability of the Docker deployment.